

Rapport de Couverture de Code et Tests Unitaires - M-Atici CRM

- **Date du rapport** : 28 mai 2026
- **Version** : 1.0.0
- **Technologie de test** : Jest (Framework de test JavaScript)
- **Couverture cible globale** : > 70%

1. Introduction

Pour garantir la fiabilité, la robustesse et la stabilité de l'application **M-Atici CRM** (spécifiquement sa partie back-end qui gère les données sensibles de facturation et de devis), une stratégie stricte de tests unitaires et d'intégration a été mise en place.

Ce rapport fournit une synthèse de la couverture du code source (Code Coverage) obtenue à l'issue de l'exécution de la suite de tests automatique.

2. Résumé de l'Exécution des Tests

L'ensemble de la suite de tests a été exécuté avec succès. Aucun échec n'a été détecté.

- **Nombre total de suites de tests** : 7 (toutes réussies)
- **Nombre total de cas de tests** : 40 (tous réussis)
- **Statut final** : **SUCCÈS (100% passés)**
- **Durée d'exécution** : 0.43 s

3. Synthèse de la Couverture de Code

Le tableau ci-dessous présente le taux de couverture des instructions, des branches décisionnelles, des fonctions et des lignes pour chaque module de l'API.

Module / Fichier	Instructions (Stmts %)	Branches (%)	Fonctions (%)	Lignes (%)
Moyenne Globale (Tous les fichiers)	73.88 %	43.90 %	77.77 %	74.19 %
<code>middlewares/authentication.middleware.js</code>	100.00 %	100.00 %	100.00 %	100.00 %
<code>services/authentication.service.js</code>	100.00 %	100.00 %	100.00 %	100.00 %
<code>services/clients.service.js</code>	100.00 %	100.00 %	100.00 %	100.00 %
<code>services/devis.service.js</code>	51.16 %	14.28 %	66.66 %	51.16 %
<code>services/email.service.js</code>	58.82 %	100.00 %	66.66 %	58.82 %
<code>services/factures.service.js</code>	63.88 %	31.25 %	66.66 %	64.70 %
<code>utils/calculDevis.js</code>	100.00 %	100.00 %	100.00 %	100.00 %

4. Détail des Modules et Couverture de Code

A. Middlewares et Services de Base (100% de couverture)

Les fichiers clés assurant la sécurité de l'application et la gestion de base des données ont été testés de manière exhaustive pour atteindre **100% de couverture de code** : * **Authentication Middleware** : Validation de la présence et de l'intégrité des tokens JWT (cas valides, tokens expirés, tokens malformés). * **Authentication Service** : Inscription, connexion, hachage sécurisé des mots de passe. * **Clients Service** : Création, modification, suppression et listage des clients avec gestion de cache Redis (y compris l'invalidation automatique du cache lors des écritures).

B. Moteurs Métiers de Calcul (100% de couverture)

- **Calcul Devis** : Les algorithmes de calcul mathématique (calcul du montant HT, application des taux de TVA variables de 20%, 10%, 5.5% et 0%, et calcul du montant TTC global) ont été validés à 100% avec des cas de tests complexes pour écarter tout risque d'erreur d'arrondi décimal.

C. Services Métiers (de 50% à 65% de couverture)

Les services plus complexes interagissant avec des ressources externes (génération de documents PDF, envoi d'emails réels via SMTP) ont fait l'objet de mocking (simulations) pour tester leur comportement logique : * **Factures Service** : Création d'une facture à partir d'un devis accepté, validation des contraintes d'unicité des numéros de facture. * **Email Service** : Envoi de devis et alertes par email (validation des formats de destinataires, pièces jointes, templates de mail). * **Devis Service** : Création, modification et suppression des devis.

5. Exécuter les Tests Localement

Pour lancer la suite de tests et afficher ce rapport de couverture dans la console, exécutez la commande suivante à la racine du projet :

```
# Pour le backend  
npm test --prefix backend -- --coverage
```

Les rapports détaillés au format HTML interactif sont également disponibles sur le serveur dans le dossier `backend/coverage/lcov-report/index.html`.